

Guia de estudio: API REST en React

Autor: Marcelo (completar si quieres nombre y apellido exacto)

Fecha: Julio 2026

1) Objetivo de esta guía

Esta guía resume y explica en profundidad la actividad de “Listado de usuarios desde una API pública” en React. Incluye:

- Fundamentos teóricos clave.
 - Explicación de cada bloque del código.
 - Ejemplos prácticos adicionales.
 - Preguntas sorpresa de profesor con respuestas.
 - Bateria tipo examen (con solución y mini justificación).
 - Un gráfico de flujo de estados para memorizar rápido.
-

2) Contexto del ejercicio

En este trabajo desarrollaste una app React que consume datos desde:

- <https://jsonplaceholder.typicode.com/users>

El flujo principal es:

1. Monta el componente.
 2. Hace fetch a la API.
 3. Gestiona estados: loading, error, usuarios.
 4. Renderiza condicionalmente según el estado.
 5. Permite recargar y filtrar por nombre.
-

3) Fundamentos teóricos esenciales

3.1 Que es una API REST

Una API REST es una interfaz que expone recursos a través de endpoints HTTP. En este caso, el recurso es la colección de usuarios.

- Recurso: users
- Endpoint: /users
- Método HTTP usado: GET

- Formato de respuesta: JSON

Idea importante: front-end y back-end se desacoplan. React no necesita saber como se guardan los datos internamente; solo consume la respuesta.

3.2 HTTP y codigos de estado

Al hacer una solicitud, el servidor responde con un status code.

- 2xx: exito
- 4xx: error del cliente (ej. 404)
- 5xx: error del servidor

Punto clave de examen: fetch NO lanza error automaticamente por 404 o 500. Solo lanza error ante fallas de red, DNS, CORS, abort, etc. Por eso hay que validar response.ok.

3.3 Estado en React

Tu componente usa estado para modelar el ciclo de vida de la carga remota:

- usuarios: array de datos.
- loading: booleano de carga.
- error: mensaje de falla.

Esto se puede pensar como una maquina de estados:

- Inicial: loading=true, error='', usuarios=[]
- Exito: loading=false, error='', usuarios=[...]
- Error: loading=false, error='mensaje', usuarios=[]

Si no separas estos estados, la UI queda confusa o inconsistente.

3.4 useEffect: cuando y por que

useEffect se usa para efectos secundarios, como fetch.

- Se ejecuta despues del render.
- Con dependencia [refreshKey], corre al montar y cada vez que cambia refreshKey.

No se recomienda declarar el callback de useEffect como async directamente, porque useEffect espera que retornes void o funcion de limpieza, no una Promise.

3.5 Limpieza de efectos y AbortController

AbortController sirve para cancelar la solicitud si el componente se desmonta. Esto evita:

- Intentar setear estado en componente desmontado.
 - Warnings de React.
 - Inconsistencias en pantallas de navegacion rapida.
-

4) Lectura guiada del código principal

4.1 Componente Usuarios

```
import { useEffect, useMemo, useState } from "react";
import UsuarioCard from "../UsuarioCard";
import "../Usuarios.css";

const API_URL = "https://jsonplaceholder.typicode.com/users";

function Usuarios() {
  const [usuarios, setUsuarios] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  const [busqueda, setBusqueda] = useState("");
  const [refreshKey, setRefreshKey] = useState(0);

  useEffect(() => {
    const abortController = new AbortController();

    const obtenerUsuarios = async () => {
      setLoading(true);
      setError("");

      try {
        const response = await fetch(API_URL, {
          signal: abortController.signal,
        });

        if (!response.ok) {
          throw new Error(
            `Error HTTP ${response.status}: no se pudo obtener la lista de usuarios.`
          );
        }

        const data = await response.json();
        setUsuarios(data);
      } catch (err) {
        if (err.name === "AbortError") {
          return;
        }

        setUsuarios([]);
        setError(
          err.message || "Ocurrió un error inesperado al cargar usuarios."
        );
      } finally {
        if (!abortController.signal.aborted) {
          setLoading(false);
        }
      }
    };
  });
}
```

```

    };

    obtenerUsuarios();

    return () => {
      abortController.abort();
    };
  }, [refreshKey]);

const usuariosFiltrados = useMemo(() => {
  const termino = busqueda.trim().toLowerCase();

  if (!termino) {
    return usuarios;
  }

  return usuarios.filter((usuario) =>
    usuario.name.toLowerCase().includes(termino),
  );
}, [usuarios, busqueda]);

const handleRecargar = () => {
  setRefreshKey((prev) => prev + 1);
};

return (
  <section className="usuarios">
    <div className="usuarios__tools">
      <label htmlFor="busqueda" className="usuarios__search-wrap">
        Buscar por nombre
        <input
          id="busqueda"
          type="search"
          placeholder="Ej: Leanne"
          value={busqueda}
          onChange={(event) => setBusqueda(event.target.value)}
        />
      </label>

      <button
        type="button"
        onClick={handleRecargar}
        className="usuarios__reload"
      >
        Recargar
      </button>
    </div>

    {loading && <p className="usuarios__status">Cargando usuarios...</p>}

    {!loading && error && (
      <div className="usuarios__error" role="alert">
        <p>{error}</p>
      </div>
    )}
  </section>
)

```

```

    })
    {!loading && !error && (
      <>
      <p className="usuarios__meta">
        Mostrando {usuariosFiltrados.length} de {usuarios.length}
      usuarios.
      </p>

      {usuariosFiltrados.length === 0 ? (
        <p className="usuarios__status">
          No hay usuarios que coincidan con la búsqueda.
        </p>
      ) : (
        <ul className="usuarios__list">
          {usuariosFiltrados.map((usuario) => (
            <UsuarioCard key={usuario.id} usuario={usuario} />
          ))}
        </ul>
      )}
    </>
  )}
</section>
);
}

```

4.2 Explicacion de decisiones importantes

- setLoading(true) al empezar una carga: evita mostrar datos viejos como si fueran actuales.
- setError("") antes de cada fetch: limpia errores anteriores.
- if (!response.ok) throw new Error(...): maneja errores HTTP explícitamente.
- catch con AbortError: no tratar cancelacion como error real.
- finally con guardia de abort: evita actualizar estado cuando no corresponde.
- useMemo en el filtro: calcula solo cuando cambia usuarios o búsqueda.
- key={usuario.id}: reconciliacion correcta de React.

4.3 Componente hijo UsuarioCard

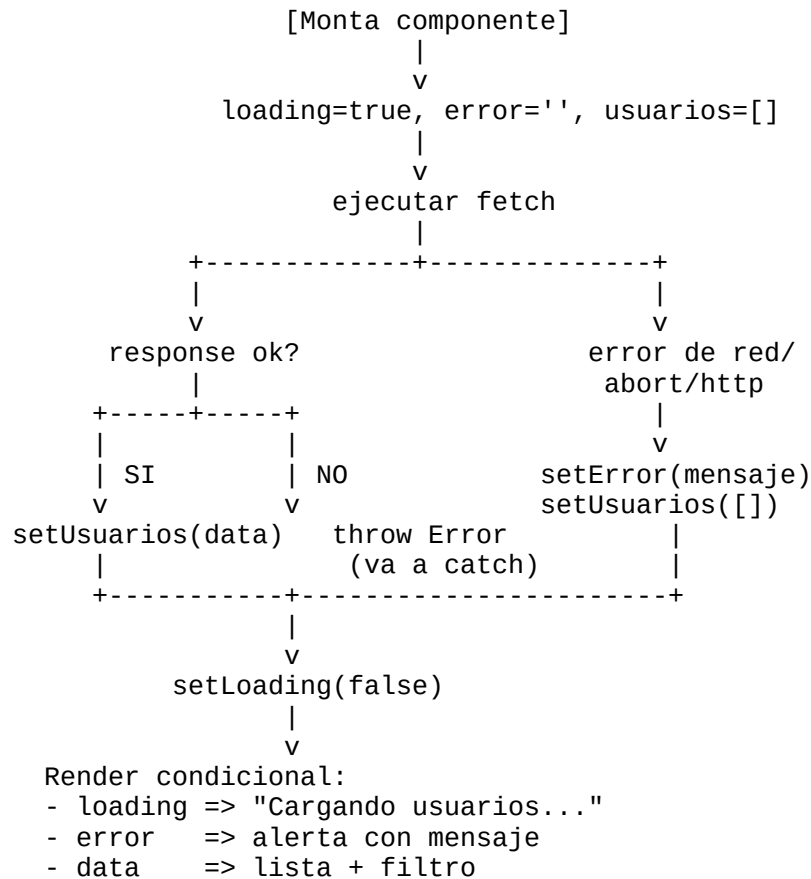
```

function UsuarioCard({ usuario }) {
  return (
    <li className="usuario-card">
      <h2>{usuario.name}</h2>
      <p>
        <strong>Email:</strong> {usuario.email}
      </p>
      <p>
        <strong>Empresa:</strong> {usuario.company?.name || "Sin dato"}
      </p>
    </li>
  );
}

```

Separa la presentacion de cada item de la logica de datos del componente padre.

5) Grafico de flujo de estados (para memorizar)



6) Ejemplos practicos adicionales

6.1 Simular error rapido (para captura de pantalla)

Cambia temporalmente la URL:

```
const API_URL = "https://jsonplaceholder.typicode.com/users-incorrecta";
```

Al recargar, deberia entrar en error con status 404.

6.2 Version minima sin extras

Si te piden una version super basica:

```
const [usuarios, setUsuarios] = useState([]);
```

```

const [loading, setLoading] = useState(true);
const [error, setError] = useState("");

useEffect(() => {
  const cargar = async () => {
    try {
      const r = await fetch("https://jsonplaceholder.typicode.com/users");
      if (!r.ok) throw new Error("Error al cargar");
      setUsuarios(await r.json());
    } catch {
      setError("No se pudo obtener usuarios");
    } finally {
      setLoading(false);
    }
  };
  cargar();
}, []);

```

6.3 Como testear mentalmente cada estado

- Estado loading: poner Network en Slow 3G en DevTools.
 - Estado error: URL incorrecta o internet off.
 - Estado exito: URL correcta + internet activa.
 - Estado lista vacia por filtro: buscar algo que no exista.
-

7) Preguntas sorpresa de profesor (con respuesta modelo)

1. Por que no haces `useEffect(async () => { ... })`? Respuesta: porque `useEffect` espera retorno void o cleanup. `Async` devuelve `Promise`.
2. `fetch` tira error con 404? Respuesta: no necesariamente; por eso valido `response.ok`.
3. Que problema evita `AbortController`? Respuesta: evita actualizar estado de un componente desmontado y permite cancelar solicitudes en curso.
4. Si saco `setError("")` antes de recargar, que podria pasar? Respuesta: puede quedar visible un error viejo durante una nueva carga.
5. Para que sirve `finally`? Respuesta: garantiza ejecutar limpieza/cierre de flujo, por ejemplo `setLoading(false)`, tanto en exito como en error.
6. Que ventaja tiene separar `UsuarioCard`? Respuesta: mejora legibilidad, reutilizacion y separacion de responsabilidades.
7. Por que `key` debe ser estable? Respuesta: para reconciliacion correcta de React y evitar errores visuales.
8. Por que `loading` arranca en `true`? Respuesta: porque al montar aun no hay datos

disponibles; la vista inicial correcta es de carga.

9. Por que usuariosFiltrados no es un useState? Respuesta: porque es estado derivado de usuarios y busqueda, no fuente primaria.
 10. Que diferencias hay entre render condicional y ocultar con CSS? Respuesta: render condicional decide si el elemento existe en el arbol React; CSS solo lo oculta visualmente.
 11. Que pasaria si response.json() falla? Respuesta: iria al catch y se mostraria error, porque tambien puede lanzar excepcion.
 12. Que tipo de errores captura catch aqui? Respuesta: excepciones de fetch, throw manual por response.ok false, y parseo JSON.
 13. Por que setUsuarios([]) en error? Respuesta: evita mostrar datos antiguos cuando la carga actual falla.
 14. Para que sirve refreshToken? Respuesta: como disparador de useEffect para forzar nueva consulta.
 15. Que significa componente controlado en el input de busqueda? Respuesta: que su valor depende del estado React (value + onChange).
-

8) Bateria tipo examen (con respuestas)

Parte A: Multiple choice

1. Cual es la razon principal para validar response.ok? A. Porque fetch lanza error si no hay JSON. B. Porque fetch no lanza error automaticamente por 4xx/5xx. C. Porque mejora el CSS. D. Porque useEffect lo exige.

Respuesta: B

2. Donde debe ir la limpieza de AbortController? A. Dentro del catch. B. En el return del useEffect. C. En el map de usuarios. D. En main.jsx.

Respuesta: B

3. Que estado representa mejor "falla al cargar"? A. loading=true y error="". B. loading=true y usuarios con datos. C. loading=false y error con mensaje. D. usuarios=null.

Respuesta: C

4. Que hace useMemo en el filtro? A. Evita re-render del componente padre. B. Memoriza un calculo derivado segun dependencias. C. Reemplaza useEffect. D. Hace fetch en cache.

Respuesta: B

5. Cual es una key recomendable al listar usuarios? A. index B. Math.random() C. usuario.id

D. usuario.name.length

Respuesta: C

Parte B: Verdadero/Falso

1. fetch siempre falla en 500. Respuesta: Falso.
2. finally se ejecuta tanto en exito como en error. Respuesta: Verdadero.
3. Un input con value y onChange es controlado. Respuesta: Verdadero.
4. Render condicional y display:none son exactamente lo mismo. Respuesta: Falso.
5. useEffect puede ejecutar efectos secundarios como llamadas HTTP. Respuesta: Verdadero.

Parte C: Desarrollo corto

1. Explica por que es util modelar loading/error/data por separado. Respuesta esperada: porque cada estado representa una fase distinta del flujo asincronico y permite una UI precisa sin contradicciones.
 2. Explica la diferencia entre error HTTP y error de red. Respuesta esperada: HTTP significa que hubo respuesta del servidor con codigo no exitoso; red significa que no pudo completarse la comunicacion o fue abortada.
 3. Explica por que se recomienda separar UsuarioCard del componente Usuarios. Respuesta esperada: por separacion de responsabilidades, mantenibilidad y legibilidad.
 4. Que problema puede causar usar index como key? Respuesta esperada: reordenamientos o renderizado incorrecto cuando cambia la lista, por identidad inestable.
 5. Que mejora haria la app mas cercana a produccion? Respuesta esperada: retries, paginacion, cache (ej. React Query), test unitarios/e2e y mensajes de error mas amigables.
-

9) Checklist de defensa oral (2-3 minutos)

- “Use useState para usuarios, loading y error porque representan fases diferentes del fetch”.
- “Use useEffect para ejecutar la solicitud al montar y al recargar”.
- “Valide response.ok porque fetch no lanza por 404/500”.
- “Manejo errores con try/catch y limpio con finally”.
- “Agregue AbortController para evitar efectos secundarios al desmontar”.
- “Implemente render condicional para mostrar loading/error/datos”.

- “Sume extras: búsqueda, recarga y componente hijo UsuarioCard”.
-

10) Mini glosario

- API REST: interfaz HTTP para recursos.
 - Endpoint: URL de un recurso.
 - JSON: formato de intercambio de datos.
 - Estado derivado: valor calculado desde estados base.
 - Render condicional: mostrar UI según una condición.
 - Cleanup: función de limpieza de un efecto.
-

11) Conclusiones

Esta actividad te hace practicar un patrón central de front-end moderno:

1. Pedir datos asíncronos.
2. Modelar estados correctamente.
3. Reflejar esos estados en la interfaz.
4. Diseñar componentes mantenibles.

Si dominas este flujo, tienes una base muy fuerte para trabajar luego con APIs reales, autenticación, paginación y librerías de data fetching.

Fin de la guía.